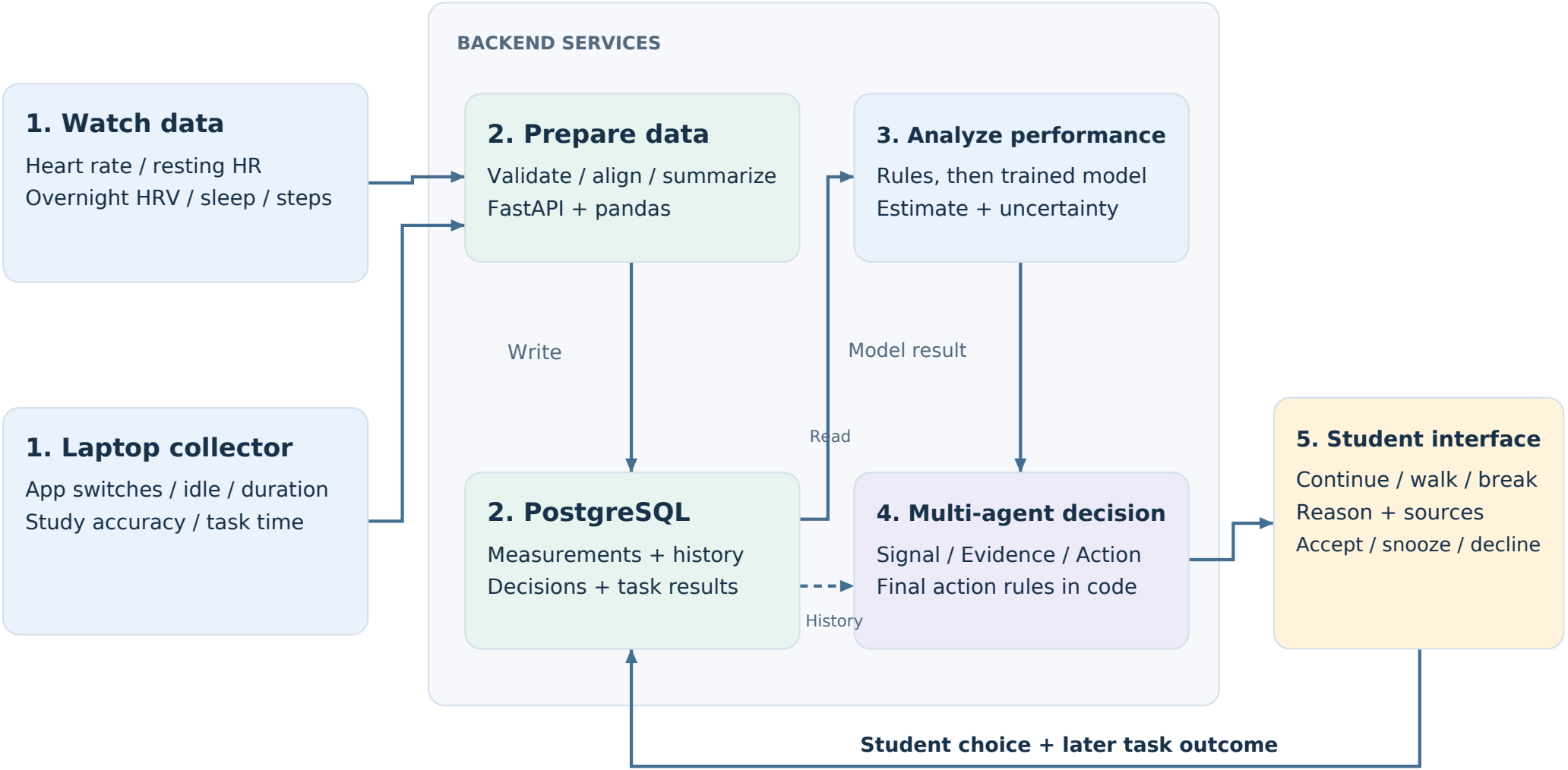
System design

One desktop app + one Python backend + one PostgreSQL database.



Consent + user-only data access + pause/delete apply across the design.

Which data will we use?

Watch = context. Laptop = behavior. Study task = measured performance.

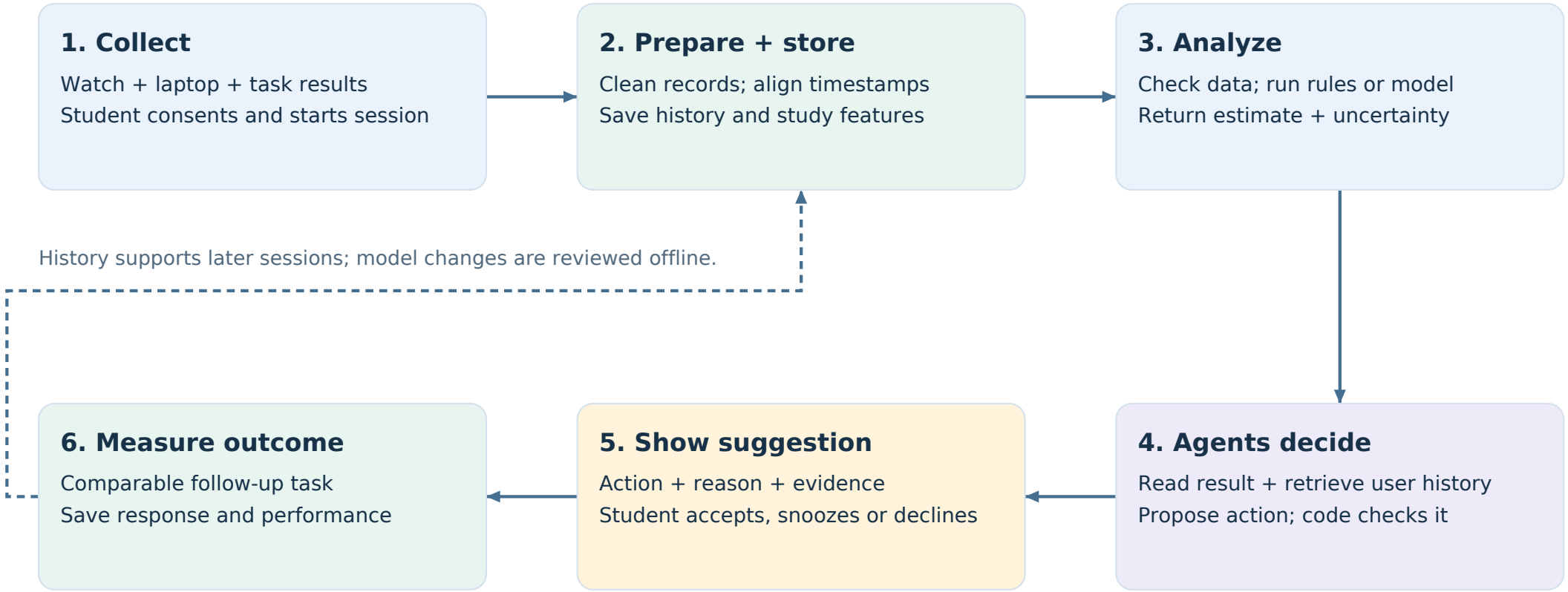
Source	Data points	What information it gives us
Watch	Heart rate + resting HR (bpm)	Physiological context; interpret current HR alongside movement.
Watch	HRV: overnight RMSSD (ms)	Change from the student's usual overnight level; daily context, not a live focus score.
Watch	Sleep duration, bedtime/wake time, awake periods	Prior-night sleep duration and schedule/continuity context.
Watch	Steps + active minutes	Movement context; distinguish a walk/exercise from sedentary study.
Laptop	Foreground app category + switch timestamps	Switches per active minute and time spent in broad app categories.
Laptop	Idle/active spans + session start/stop	Idle fraction and continuous study duration; idle does not always mean distraction.
Study task	Task version/difficulty, correctness, response/completion time	Independent performance labels to test predictions and intervention benefit.
User check-in	Fatigue rating + chosen/completed break	Optional subjective context and adherence; not a substitute for task labels.

Watch fields depend on the chosen device, permissions and sync. Use overnight HRV summaries initially; do not assume continuous daytime HRV. [S8-S10]

Phone signals and CGM are optional future experiments. The first build runs without them.

Step-by-step flow

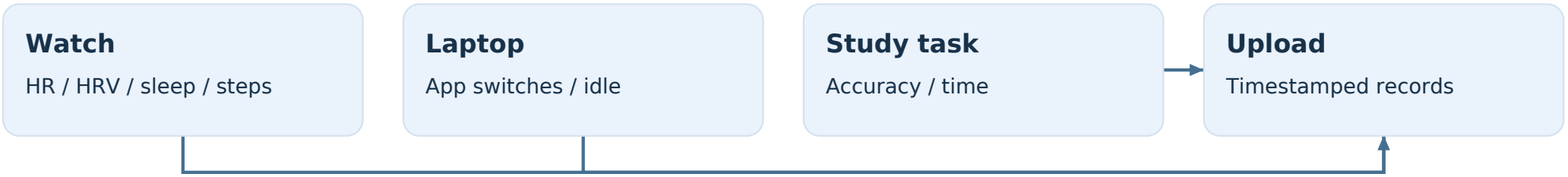
Follow one study session from collection to a measured outcome.



If essential data is missing: stop automatic personalization and show "Not enough data". Keep manual break controls available.

1. Collect data

Capture context and outcomes without collecting private content.



1. Watch context

Read heart rate, daily resting HR, overnight HRV (RMSSD), sleep and steps through permitted Google Health access or participant exports. Show sync time; these are not guaranteed live readings. [S8-S11]

2. Laptop behavior

Use Electron for idle time and an OS-specific adapter for foreground-app categories. Derive switches and study duration. No typed text, window titles or screenshots. [S1, S12, S15]

3. Actual performance

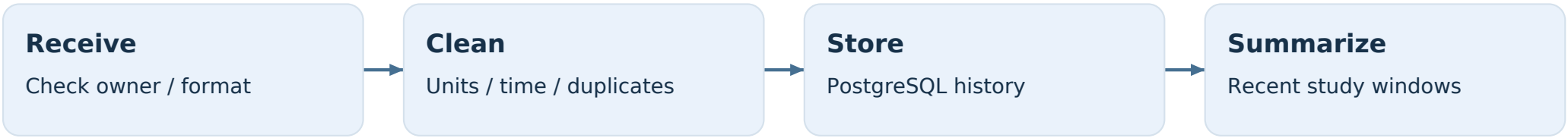
Record task version, difficulty, correct answers and response/completion time. These independent outcomes let us test whether the sensors add value. Optional fatigue ratings add user context.

4. Why this component is needed

Combine different sources in records with user, session, metric, value, unit, measured time and received time. Pause collection on request; retain unsent batches locally until the server confirms them.

2. Prepare and store data

Create reliable, time-aligned records that every later step can reuse.



1. Validate and save

FastAPI/Pydantic checks input shape and consent; PostgreSQL stores measurements, sessions, decisions and outcomes. Deduplicate repeated uploads. Every read is restricted to the signed-in user. [S4, S14]

2. Align the signals

Use pandas to summarize past 5- and 15-minute study windows. Attach sleep and overnight HRV as daily context. Preserve missing values; use only data that had arrived before the recommendation. [S13]

3. Background processing

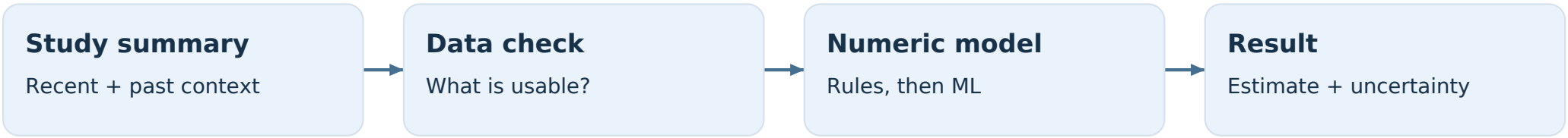
A small Python task runner reads a saved to-do list in PostgreSQL. It handles slow wearable imports and retries failed work so the screen stays responsive. Save tasks with their data; repeating a task must not duplicate its result.

4. Why this component is needed

It prevents duplicate, stale or mismatched measurements from becoming model inputs. Output one clean study summary: behavior metrics, available watch context, timestamps and missing-data flags. No separate queue service is needed initially.

3. Analyze performance

Estimate a defined task outcome instead of inventing a universal productivity score.



1. Calculate a few features

Start with app switches per active minute, idle fraction, session duration and recent task accuracy. Test whether sleep, overnight HRV and movement-adjusted HR improve prediction beyond laptop data.

2. Use a testable model

Begin with transparent rules; fit a regularized scikit-learn model after collecting independent task labels. Predict future task accuracy or time. Test on later sessions; compare laptop-only and wearable-enhanced models. [S6]

3. Data completeness check

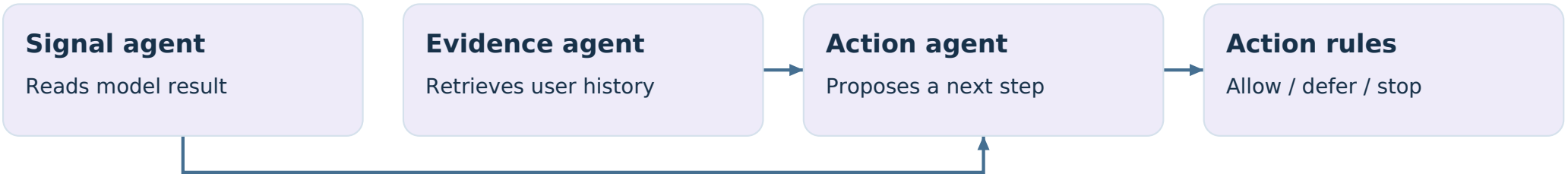
This simply asks whether enough recent, valid data exists. If watch data is absent, use a validated laptop-only model. If essential session data is missing, show "Not enough data" instead of a confident recommendation.

4. Why this component is needed

It turns measurements into a reproducible estimate with a model version and uncertainty. Numerical code calculates the result; an LLM does not infer a fatigue score directly from HRV. Small pilots can stay in rule-based mode.

4. Choose a recommendation

This is where the multi-agent model is used.



1. Signal agent

Reads the numeric estimate, recent features and missing-data notes from component 3. It can call the scoring tool; it cannot create missing measurements or a medical diagnosis.

2. Evidence agent

Retrieves relevant past sessions, preferences and outcomes from PostgreSQL. This supplies source-backed context for the LLM (retrieval-augmented generation, or RAG). Start with SQL retrieval; semantic vector search can wait.

3. Action agent and final rules

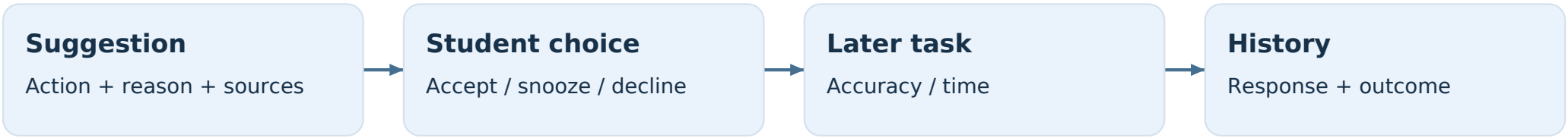
Proposes continue, a short walk or a quiet break. Ordinary code checks consent, recent prompts and user preferences before delivery. The LLM cannot override those checks.

4. Why this component is needed

It combines the current estimate with personal history and explains a suggestion using cited records. LangGraph coordinates the three roles and tool calls; one LLM can serve all roles. No separate agent servers are needed. [S7]

5. Show advice and measure the result

Give the student control and close the feedback loop.



1. One desktop interface

Use React inside the same Electron app that collects laptop data. Show start/pause, source status, the recommendation, evidence and uncertainty. A separate Next.js website is not needed for the first build. [S1-S2]

2. Record the user choice

Keep accept, snooze and decline separate from actual break completion. Use a stable recommendation ID so retries do not produce duplicate prompts. A declined suggestion is a valid response.

3. Measure later performance

Record a comparable follow-up task. Report accuracy and speed separately; include break time in fixed-wall-clock comparisons. Missing follow-up remains missing. Save the response and outcome in PostgreSQL.

4. Why this component is needed

It tests usefulness rather than just displaying a prediction. Compare adaptive and fixed-break schedules using randomized sessions/blocks. Freeze the evaluated policy; update models offline after reviewing results.

Recommended stack: what is actually needed

These are project-fit choices. Industry use supports maturity; it does not prove prediction accuracy.

Tool	Use it when	Why it fits this project	Verified industry evidence
Electron + TypeScript	Desktop build	Idle time, local notifications and desktop integration. Foreground-app capture still needs an OS adapter.	VS Code / Slack [S1]
React	Desktop UI	One interface for sources, suggestions and response controls; reuse it inside Electron.	Meta production surfaces [S2]
FastAPI + Pydantic	Backend build	Keep data validation, API endpoints and Python analysis together; reject malformed sensor records.	FastAPI used in Uber Ludwig; Pydantic powers its validation [S3, S14]
PostgreSQL	First saved session	One database for measurements, personal history, outcomes and saved background tasks.	GitLab database support [S4]
pandas	Feature pipeline	Time alignment, rolling summaries and missing data; sufficient for a small class dataset.	AWS SDK for pandas [S5, S13]
scikit-learn	After task labels	Regularized models and reproducible evaluation for a few structured features. Rules work before enough labels exist.	Spotify testimonial [S6]
LangGraph + one LLM API	Multi-agent demo	Coordinates the three roles and tool calls. Useful for explicit traces; not required for the numerical model itself.	Vodafone customer case [S7]

Start with one desktop app, one backend and one database. Use SQL history retrieval; a separate vector database, queue platform and Kubernetes are unnecessary for the initial pilot.

Development order

Complete each proof before adding the next layer.

#	Build	Work to do	Completion check
1	Define the task + prove access	Choose one repeatable study task; confirm watch fields and consent.	One task result + one permitted watch record.
2	Collect + save data	Build Electron capture, FastAPI upload and PostgreSQL storage.	Pause, reconnect and duplicate upload work correctly.
3	Clean + summarize	Build pandas features and a simple fixed/manual-break baseline.	No future data; missing sources remain visible.
4	Test the numeric model	Train on task labels; compare laptop-only with added watch context.	Later-session results, calibration and limitations recorded.
5	Connect agents + interface	Add LangGraph roles, SQL evidence and action rules; show citations.	One traceable suggestion and recorded user response.
6	Evaluate + demonstrate	Randomize adaptive vs fixed-break sessions; test failure paths.	Outcome estimates, uncertainty, burden and recorded fallback.

The two tests are different: does watch data improve prediction, and do adaptive breaks improve task outcomes? A negative result should simplify the product.

Verified primary sources

Checked October 15, 2026. Company examples are documented use cases, not a market-share ranking.

S1	**Electron: documented desktop adopters**	https://www.electronjs.org/docs/latest/why-electron
S2	**React: production use at Meta**	https://react.dev/
S3	**Uber: FastAPI in Ludwig**	https://www.uber.com/us/en/blog/ludwig-v0-2/
S4	**GitLab: PostgreSQL database support**	https://docs.gitlab.com/omnibus/settings/database/
S5	**AWS SDK for pandas**	https://github.com/aws/aws-sdk-pandas
S6	**scikit-learn: Spotify testimonial**	https://scikit-learn.org/stable/testimonials/testimonials.html
S7	**LangGraph: Vodafone customer case**	https://www.langchain.com/blog/customers-vodafone
S8	**Google Health: data types and sync**	https://developers.google.com/health/data-types
S9	**Google Health: sleep / overnight HRV**	https://developers.google.com/health/data-types/sleep
S10	**Google Health: HR / RMSSD fields**	https://developers.google.com/health/data-types/vitals
S11	**Google Health: current developer setup**	https://developers.google.com/health/get-started
S12	**Electron: idle time / sleep-wake APIs**	https://www.electronjs.org/docs/latest/api/power-monitor
S13	**pandas: time alignment / resampling**	https://pandas.pydata.org/docs/user_guide/timeseries.html
S14	**FastAPI: validation / Pydantic**	https://fastapi.tiangolo.com/
S15	**Apple: foreground application API**	https://developer.apple.com/documentation/appkit/nsworkspace/frontmostapplication

Google now provides new-developer setup instructions. Confirm project permissions and device access; use permitted exports/replay when needed. This replaces the earlier blanket onboarding-closed assumption. [S11]